BÁO CÁO KỸ THUẬT

KHẢO SÁT KHÔNG GIAN THIẾT KẾ BỘ NHỚ ĐỆM L2 CHIA SẺ BẰNG GEM5

Sinh viên thực hiện: Huỳnh Vũ Thục Quyên

MSSV: 24200163

Mục lục

1 Tổng quan đề tài **3**

2 Cơ sở lý thuyết và kiến trúc hệ thống **3**
2.1 Cache . . . 3
2.2 L1 và L2 . . . 3
2.2.1 L1 Cache . . . 4
2.2.2 L2 Cache . . . 4
2.3 Set, Way và Cache Line . . . 4
2.3.1 Cache Line . . . 4
2.3.2 Set . . . 4
2.3.3 Way . . . 5
2.4 Cache Hit và Cache Miss . . . 5
2.5 Interconnect và NoC . . . 5
2.6 Memory Hierarchy . . . 6
2.7 MinorCPU trong gem5 . . . 6
2.8 Ghép thành hệ thống . . . 6
2.9 Workload và mô hình traffic . . . 6
2.10 Chỉ số Miss Rate . . . 7
2.11 Chỉ số AMAT . . . 7
2.12 Way Shutdown . . . 8
2.13 MSHR . . . 8

3 Mục tiêu và ý nghĩa của bài toán **9**
3.1 Cơ sở lựa chọn shared L2 . . . 9
3.2 Ý nghĩa của việc khảo sát active ways . . . 9
3.3 Vai trò của stride và footprint . . . 10
3.4 Vai trò của Miss Rate trong đánh giá . . . 10
3.5 Vai trò của AMAT trong đánh giá . . . 10
3.6 Vai trò của MSHR trong hệ thống . . . 10

4 Phạm vi và điều kiện áp dụng **10**
4.1 Điều kiện áp dụng shared cache . . . 10
4.2 Điều kiện áp dụng Way Shutdown . . . 11
4.3 Điều kiện khảo sát Capacity . . . 11
4.4 Điều kiện cần xem xét MSHR . . . 11

5 Đối tượng và vai trò liên quan **11**

6 Quy trình triển khai và đánh giá **11**
6.1 Xác định kiến trúc baseline . . . 11
6.2 Thiết lập các RiscvMinorCPU . . . 12
6.3 Thiết lập private L1 cho từng core . . . 12
6.4 Thiết lập shared L2 . . . 12
6.5 Kết nối L1 với interconnect . . . 13
6.6 Kết nối L2 với Memory Controller và DRAM . . . 13
6.7 Thiết lập các tham số hệ thống . . . 13
6.8 Thiết lập workload . . . 14
6.9 Kiểm tra baseline trước khi DSE . . . 14
6.10 Cơ chế hoạt động của MSHR . . . 14

6.11 Thiết lập MSHR baseline 15
6.12 Thực nghiệm Way Shutdown 15
6.13 Khảo sát Capacity 16
6.14 Khảo sát Associativity 16
6.15 Thu thập và xử lý kết quả mô phỏng 16
6.16 Tổng hợp kết quả Miss Rate 17
6.17 Phân tích Miss Rate 18
6.18 Tổng hợp kết quả AMAT 18
6.19 So sánh hiệu năng 19
6.20 Phân tích hiệu năng 19
6.21 Tổ chức và trình bày kết quả 19
6.22 Giới hạn của mô hình 20

1 Tổng quan đề tài

Đề tài tập trung vào việc khảo sát không gian thiết kế của **shared L2 cache** trong một hệ thống multicore RISC-V bằng gem5.

Theo yêu cầu đề tài, hệ thống được xây dựng với bốn core RISC-V in-order sử dụng `MinorCPU`. Mỗi core có private L1 cache và bốn core cùng truy cập một shared L2 cache.

Mô hình tổng quát:

$$4 \times RISC\text{-}V\ CPU \rightarrow Private\ L1 \rightarrow Interconnect \rightarrow Shared\ L2 \rightarrow Memory \quad (1)$$

Các tham số của L2 được khảo sát theo thứ tự ưu tiên:

1. Số way hoạt động, phục vụ đánh giá phương án *way shutdown*;
2. Dung lượng L2;
3. Associativity.

Cache line được cố định ở 64 byte và baseline sử dụng 16-way.

Traffic vector/DSP được xấp xỉ bằng synthetic traffic generator với các đặc trưng như *stride* và *footprint*.

MSHR được nghiên cứu bổ sung. Theo cấu hình baseline dùng để đánh giá, số MSHR được đặt là 32 và được giữ cố định trong các thí nghiệm chính khi khảo sát active ways.

2 Cơ sở lý thuyết và kiến trúc hệ thống

Phần này trình bày các khái niệm nền tảng, kiến trúc baseline và các thành phần được sử dụng trong mô hình mô phỏng. Nội dung được sắp xếp từ memory hierarchy, cache, interconnect đến CPU model, workload và các chỉ số đánh giá hiệu năng.

2.1 Cache

Cache (bộ nhớ đệm) là một vùng lưu trữ dữ liệu tạm thời có tốc độ truy xuất cao hơn và nằm giữa CPU với bộ nhớ chính.

Thay vì CPU luôn truy cập DRAM, dữ liệu thường xuyên được sử dụng có thể được lưu trong các cache gần CPU hơn, từ đó giúp giảm độ trễ truy cập và cải thiện hiệu năng xử lý.

Một memory access có thể đi qua:

$$CPU \rightarrow L1 \rightarrow L2 \rightarrow DRAM \quad (2)$$

nếu dữ liệu không được tìm thấy ở các tầng cache phía trên.

2.2 L1 và L2

Cache được tổ chức thành nhiều mức.

2.2.1 L1 Cache

L1 là cache gần CPU nhất và thường có latency thấp.

L1 là private cache. Mỗi core có một L1 riêng:

$$Core_i \rightarrow Private\ L1_i \tag{3}$$

2.2.2 L2 Cache

L2 nằm phía dưới L1.

Trong baseline của project, L2 được tổ chức dưới dạng shared cache:

$$L1_0, L1_1, L1_2, L1_3 \rightarrow Shared\ L2 \tag{4}$$

Shared L2 cho phép bốn core cùng sử dụng một tài nguyên cache.

2.3 Set, Way và Cache Line

2.3.1 Cache Line

Cache line là đơn vị lưu trữ và truyền dữ liệu cơ bản của cache. Trong một set, mỗi way có thể chứa một cache line.

Một cache line có thể bao gồm các thành phần chính:

- **Valid bit**: bit cho biết dữ liệu trong cache line có hợp lệ hay không;
- **Dirty bit**: bit cho biết dữ liệu trong cache line đã bị sửa đổi so với tầng memory phía dưới hay chưa;
- **Tag**: lưu các bit địa chỉ dùng để xác định cache line đang chứa dữ liệu của vùng địa chỉ nào;
- **Data Block**: khối dữ liệu thực tế được lưu trong cache line. Với cache line 64 Byte và word 32 bit, một cache line tương ứng với 16 words.

2.3.2 Set

Cache được chia thành nhiều set.

Địa chỉ bộ nhớ được phân chia thành các trường như:

$$Address = Tag + Set\ Index + Byte\ Offset \tag{5}$$

Với cache line có kích thước 64 Byte:

$$Byte\ Offset = \log_2(64) = 6\ bits \tag{6}$$

Số bit dùng cho *Set Index* phụ thuộc vào tổng số set của cache.

2.3.3 Way

Mỗi set có nhiều vị trí có thể chứa cache line. Các vị trí này được gọi là way.

Nếu cache là 16-way:

$$16\ ways/set \tag{7}$$

Do đó:

$$Associativity = Number\ of\ Ways\ per\ Set \tag{8}$$

2.4 Cache Hit và Cache Miss

Khi CPU yêu cầu một cache line:

- Nếu cache line đã tồn tại trong cache: **cache hit**;
- Nếu cache line không tồn tại: **cache miss**.

Nếu L1 miss, request có thể tiếp tục xuống L2.

Nếu L2 cũng miss, request tiếp tục xuống DRAM.

Do đó:

$$L1\ Miss \rightarrow L2\ Access \tag{9}$$

và:

$$L2\ Miss \rightarrow DRAM\ Access \tag{10}$$

2.5 Interconnect và NoC

Interconnect là thành phần chịu trách nhiệm kết nối và truyền request/response giữa các khối trong hệ thống.

Trong kiến trúc baseline của project, các private L1 được kết nối tới shared L2 thông qua một interconnect:

$$Private\ L1 \rightarrow Interconnect \rightarrow Shared\ L2 \tag{11}$$

NoC (*Network-on-Chip*) là một kiến trúc interconnect tổng quát, trong đó các thành phần trên chip giao tiếp thông qua mạng gồm các node, link và router.

Trong baseline của project, interconnect được hiện thực bằng component `L2XBar`. `L2XBar` là thành phần kết nối các L1 với shared L2 trong mô hình gem5 được sử dụng.

2.6 Memory Hierarchy

Một hierarchy đơn giản của project:

CPU Registers
↓
L1
↓
Shared L2
↓
DRAM

Các tầng phía trên thường có latency thấp hơn nhưng capacity nhỏ hơn.

Các tầng phía dưới thường có capacity lớn hơn nhưng latency cao hơn. Do sự phân cấp này, cấu trúc trên được gọi là *memory hierarchy*.

Khi một access miss ở một tầng, request có thể tiếp tục xuống tầng thấp hơn.

2.7 MinorCPU trong gem5

Trong project, CPU model được sử dụng là:

`RiscvMinorCPU (do gem5 cung cấp)`

`MinorCPU` là CPU model in-order.

Một instance của `RiscvMinorCPU` đại diện cho một core trong mô hình mô phỏng.

2.8 Ghép thành hệ thống

Các thành phần được ghép thành baseline:

RiscvMinorCPU 0 → Private L1
RiscvMinorCPU 1 → Private L1
RiscvMinorCPU 2 → Private L1
RiscvMinorCPU 3 → Private L1
↓
Interconnect
↓
Shared L2
↓
Memory Controller
↓
DDR4 / DRAM

2.9 Workload và mô hình traffic

Workload là chương trình hoặc traffic pattern tạo ra memory access cho hệ thống.

Project sử dụng synthetic traffic có đặc tính gần với vector/DSP.

Hai tham số chính:

- *stride*: khoảng cách giữa các địa chỉ liên tiếp;
- *footprint*: phạm vi dữ liệu được workload sử dụng.

Ví dụ baseline:

$$Stride = 64\ bytes \tag{12}$$

$$Footprint = 2\ MB \tag{13}$$

Footprint được chọn lớn hơn L1 để working set không nằm hoàn toàn trong L1.

Tuy nhiên:

$$Footprint > L1 \tag{14}$$

không đồng nghĩa mọi access đều là L1 miss.

Miss rate vẫn phụ thuộc vào locality, stride và chuỗi địa chỉ.

2.10 Chỉ số Miss Rate

Miss rate là tỷ lệ access không tìm thấy dữ liệu trong cache.

$$MissRate = \frac{Number\ of\ Misses}{Number\ of\ Accesses} \tag{15}$$

Ví dụ nếu có 1,000 L2 accesses và 100 L2 misses thì miss rate bằng 10%.

Trong project, miss rate là một trong các output chính được dùng để so sánh các configuration.

2.11 Chỉ số AMAT

AMAT (*Average Memory Access Time*) là thời gian truy cập bộ nhớ trung bình, được sử dụng để đánh giá tác động tổng hợp của hit time và chi phí do cache miss trong memory hierarchy.

Công thức tổng quát:

$$AMAT = HitTime + MissRate \times MissPenalty \tag{16}$$

Đối với hệ thống phân cấp L1–L2–DRAM, AMAT cần được xác định theo từng cấp.

Ở cấp L2:

$$AMAT_{L2} = T_{L2} + MR_{L2} \times P_{L2} \tag{17}$$

trong đó:

- T_{L2} là thời gian truy cập L2;
- MR_{L2} là L2 miss rate;
- P_{L2} là miss penalty khi L2 miss.

Đối với toàn bộ đường truy cập từ CPU:

$$AMAT_{system} = T_{L1} + MR_{L1} \times (T_{L2} + MR_{L2} \times P_{L2}) \tag{18}$$

Trong đó:

- T_{L1}: thời gian truy cập L1;
- MR_{L1}: miss rate của L1;
- T_{L2}: thời gian xử lý một access tại L2, bao gồm latency của đường interconnect nếu cách đo được định nghĩa như vậy;
- MR_{L2}: miss rate của L2;
- P_{L2}: chi phí bổ sung khi L2 miss và phải truy cập memory system phía dưới.

Các thành phần latency phải được xác định theo cùng một phạm vi đo. Không được cộng thêm một thành phần latency nếu thành phần đó đã được bao gồm trong đại lượng đang sử dụng.

2.12 Way Shutdown

Way shutdown là phương án giảm số way đang hoạt động trong cache để khảo sát trade-off giữa tài nguyên cache và hiệu năng.

Baseline:

$$Associativity = 16 \tag{19}$$

Sau đó khảo sát các cấu hình:

$$16,\ 12,\ 8,\ 4,\ 2 \tag{20}$$

Cần phân biệt:

- **Associativity**: số way được thiết kế trong mỗi set;
- **Active ways**: số way được phép hoạt động trong configuration hiện tại.

2.13 MSHR

MSHR là viết tắt của *Miss Status Holding Register*.

MSHR được sử dụng trong non-blocking cache để theo dõi các cache miss đang chờ dữ liệu trả về từ tầng memory thấp hơn.

Một quy trình đơn giản:

CPU request

↓

Cache lookup

↓

Hit → trả dữ liệu

Miss

↓

Tạo / sử dụng MSHR entry

↓

Gửi request xuống lower-level cache hoặc memory

↓

Chờ dữ liệu trả về

↓

Hoàn tất outstanding miss

Nếu có nhiều miss xảy ra đồng thời, nhiều MSHR entry có thể được sử dụng.

Vì vậy số lượng MSHR liên quan đến khả năng theo dõi nhiều outstanding misses và khả năng khai thác memory-level parallelism.

3 Mục tiêu và ý nghĩa của bài toán

Trong hệ thống multicore, các tài nguyên dùng chung trong memory hierarchy có thể tạo ra inter-core interference, làm thay đổi timing và performance của các task.

Do đó, không thể chỉ xem CPU như một tài nguyên độc lập mà cần quan tâm đến các shared resource trong memory hierarchy.

3.1 Cơ sở lựa chọn shared L2

Trong multicore, shared L2 là tài nguyên được nhiều core cùng truy cập.

Điều này tạo ra hai vấn đề:

1. Các core phải cạnh tranh tài nguyên cache;
2. Access pattern của một core có thể ảnh hưởng đến cache behavior của core khác.

3.2 Ý nghĩa của việc khảo sát active ways

Cache nhiều way cung cấp nhiều vị trí lưu trữ hơn trong mỗi set, nhưng cũng đồng nghĩa với việc sử dụng nhiều tài nguyên cache hơn.

Nếu giảm số active ways, có thể giảm tài nguyên đang hoạt động, nhưng nguy cơ tăng conflict hoặc capacity-related miss cũng có thể tăng.

Vì vậy cần khảo sát:

$$16 \rightarrow 12 \rightarrow 8 \rightarrow 4 \rightarrow 2 \tag{21}$$

để tìm trade-off giữa tài nguyên cache và hiệu năng.

3.3 Vai trò của stride và footprint

Cache behavior phụ thuộc mạnh vào chuỗi địa chỉ truy cập.

Do đó, synthetic traffic cho phép kiểm soát các đặc tính của workload như:

- stride;
- footprint;
- locality;
- mức độ memory traffic.

Điều này giúp so sánh công bằng giữa các configuration.

3.4 Vai trò của Miss Rate trong đánh giá

Miss rate cho biết mức độ hiệu quả của cache đối với workload.

Khi thay đổi active ways, capacity hoặc associativity, miss rate cho biết cache behavior đã thay đổi như thế nào.

Do đó miss rate là output trực tiếp quan trọng của DSE.

3.5 Vai trò của AMAT trong đánh giá

Miss rate một mình chưa thể hiện đầy đủ performance.

Hai configuration có thể có miss rate khác nhau nhưng miss penalty cũng khác nhau.

AMAT kết hợp:

$$HitTime + MissRate \times MissPenalty \tag{22}$$

để tạo ra một chỉ số về average memory access time.

3.6 Vai trò của MSHR trong hệ thống

MSHR liên quan đến khả năng xử lý nhiều cache miss đang outstanding.

Nếu số MSHR trong shared non-blocking cache không đủ, MSHR contention có thể trở thành nguồn inter-core interference.

Do đó, dù MSHR nằm ngoài phạm vi mô phỏng chính ban đầu, việc hiểu MSHR giúp giải thích tại sao một workload có nhiều memory request đồng thời có thể bị giới hạn.

4 Phạm vi và điều kiện áp dụng

4.1 Điều kiện áp dụng shared cache

Cần khảo sát shared cache khi hệ thống có nhiều core cùng truy cập một cache cấp dưới.

Đặc biệt, shared cache quan trọng khi muốn nghiên cứu:

- cache contention;
- cache interference;

- cache partitioning;
- cache capacity;
- associativity;
- way allocation.

4.2 Điều kiện áp dụng Way Shutdown

Way shutdown phù hợp khi mục tiêu là khảo sát việc giảm số way hoạt động và đánh đổi với performance.

4.3 Điều kiện khảo sát Capacity

Capacity được khảo sát khi cần đánh giá ảnh hưởng của tổng dung lượng L2 đến khả năng chứa working set.

Nếu working set lớn hơn cache, cache có thể chịu nhiều eviction hơn.

4.4 Điều kiện cần xem xét MSHR

MSHR đặc biệt quan trọng khi workload tạo ra nhiều outstanding memory requests.

Nếu số MSHR không đủ, cache có thể không tiếp tục nhận thêm miss mới dù memory system phía dưới vẫn còn khả năng xử lý.

5 Đối tượng và vai trò liên quan

Các vấn đề được khảo sát thuộc lĩnh vực computer architecture và memory system.

Các nhóm vai trò liên quan gồm:

- SoC Architect;
- Computer Architecture Engineer;
- Cache / Memory System Architect;
- Performance Engineer;
- DSE Engineer;
- Simulation Engineer;
- Systems Researcher.

6 Quy trình triển khai và đánh giá

6.1 Xác định kiến trúc baseline

Baseline trước tiên được xác định:

$$4\times RiscvMinorCPU \rightarrow Private\,L1 \rightarrow Interconnect \rightarrow Shared\,L2 \rightarrow Memory\,Controller \rightarrow DRA \tag{23}$$

Mục tiêu của baseline là tạo một hệ thống ổn định có thể chạy workload trước khi bắt đầu DSE.

6.2 Thiết lập các RiscvMinorCPU

Bốn CPU model được tạo.

Mỗi:

`RiscvMinorCPU`

được xem là một core.

Do đó:

$$Core_0,\ Core_1,\ Core_2,\ Core_3 \tag{24}$$

được tạo từ bốn instance của CPU model.

6.3 Thiết lập private L1 cho từng core

Mỗi core được gắn:

- một instruction cache;
- một data cache.

Baseline theo skeleton:

Bảng 1: Private L1 baseline

Thông số	Giá trị
I-cache	32 KB
D-cache	32 KB
Associativity	4-way
Latency	1 cycle
Kiểu	Private per core

6.4 Thiết lập shared L2

Một L2 duy nhất được tạo cho cả bốn core.

Baseline:

Bảng 2: Shared L2 baseline

Thông số	Giá trị
Cache line	64 bytes
Capacity	1 MB theo skeleton
Associativity	16-way
Hit latency	14 cycles theo skeleton
MSHR	32

Trong baseline chính:

$$L2 = 16\text{-}way \tag{25}$$

và:

$$MSHR = 32 \tag{26}$$

6.5 Kết nối L1 với interconnect

Các L1 không nối trực tiếp với L2 mà đi qua interconnect.

Mô hình:

$$L1_0, L1_1, L1_2, L1_3 \rightarrow Interconnect \rightarrow L2 \tag{27}$$

Skeleton sử dụng `L2XBar`.

Các thông số:

Bảng 3: Interconnect baseline

Thông số	Giá trị
Component	L2XBar
Width	16 bytes
Forward latency	2 cycles
Response latency	2 cycles

6.6 Kết nối L2 với Memory Controller và DRAM

L2 được nối tới memory system:

$$L2 \rightarrow SystemXBar \rightarrow MemCtrl \rightarrow DDR4 \tag{28}$$

Mục tiêu là để khi L2 miss, request có thể tiếp tục xuống memory.

6.7 Thiết lập các tham số hệ thống

Baseline system:

Bảng 4: System parameters

Thông số	Giá trị
Number of cores	4
CPU model	RiscvMinorCPU
Clock	1 GHz
Memory mode	Timing
Memory size	512 MB
Cache line	64 bytes

6.8 Thiết lập workload

Synthetic workload được cấu hình với:

$$Stride = 64\ bytes \tag{29}$$

và:

$$Footprint = 2\ MB \tag{30}$$

Footprint phải lớn hơn L1 nhằm đảm bảo working set vượt capacity của L1.

Workload được giữ giống nhau giữa các configuration để việc so sánh có ý nghĩa.

6.9 Kiểm tra baseline trước khi DSE

Trước tiên cần chạy baseline để kiểm tra:

1. gem5 khởi tạo thành công;
2. CPU chạy được workload;
3. L1 hoạt động;
4. L2 nhận được request;
5. memory controller nhận được request;
6. simulation kết thúc bình thường;
7. statistics được sinh ra.

Baseline là mốc tham chiếu cho toàn bộ configuration phía sau.

6.10 Cơ chế hoạt động của MSHR

Khi CPU gửi một request:

$$CPU \rightarrow Cache\ lookup \tag{31}$$

Nếu hit:

$$Hit \rightarrow Return\ data \tag{32}$$

Nếu miss:

$$Miss \rightarrow Allocate/Use\ MSHR \rightarrow Send\ request\ downward \tag{33}$$

MSHR giữ trạng thái của miss đang outstanding.

Khi lower-level memory trả dữ liệu:

$$Data\ Return \rightarrow MSHR\ Completion \rightarrow Cache\ Update \rightarrow Request\ Completion \tag{34}$$

Nếu nhiều miss xảy ra đồng thời:

$$Miss_1, Miss_2, \ldots, Miss_n \tag{35}$$

thì cần đủ MSHR entry để theo dõi các miss đó.

Do đó:

$$MSHR\ Capacity \rightarrow Outstanding\ Misses \rightarrow Memory\ Level\ Parallelism \tag{36}$$

Nếu MSHR bị đầy, request mới có thể phải chờ.

6.11 Thiết lập MSHR baseline

Baseline sử dụng:

$$MSHR = 32 \tag{37}$$

Trong khảo sát way shutdown:

$$MSHR = 32 \tag{38}$$

được giữ cố định.

Mục đích là chỉ thay đổi một biến chính:

$$ActiveWays \tag{39}$$

thay vì đồng thời thay đổi MSHR.

6.12 Thực nghiệm Way Shutdown

Baseline:

$$ActiveWays = 16 \tag{40}$$

Sau đó chạy:

$$16,\ 12,\ 8,\ 4,\ 2 \tag{41}$$

Trong mỗi configuration:

Bảng 5: Các tham số trong thí nghiệm way shutdown

Thông số	Baseline	Way shutdown
Line size	64 B	Fixed
L2 capacity	1 MB	Fixed
Associativity	16-way	Fixed
MSHR	32	Fixed
Stride	64 B	Fixed
Footprint	2 MB	Fixed
Active ways	16	16/12/8/4/2

Mục tiêu là xác định mức suy giảm performance khi giảm số way hoạt động.

6.13 Khảo sát Capacity

Sau P1, capacity được thay đổi.

Các yếu tố khác được giữ cố định trong từng thí nghiệm.

Mục tiêu:

$$Capacity \rightarrow Cache\ space \rightarrow Miss\ behavior \rightarrow Performance \tag{42}$$

Kết quả cần được so sánh với baseline.

6.14 Khảo sát Associativity

Sau capacity, associativity được khảo sát.

Mục tiêu là xác định ảnh hưởng của số way trên mỗi set đến conflict miss và overall cache behavior.

Lưu ý:

$$Associativity \neq ActiveWays \tag{43}$$

Associativity là đặc tính của cache geometry.

Active ways là số way được phép hoạt động trong configuration.

6.15 Thu thập và xử lý kết quả mô phỏng

Sau mỗi simulation, statistics được thu thập.

Quy trình:

Configuration

↓

gem5 Simulation

↓

Statistics

↓

Extract Metrics

↓

Data Processing

↓

Performance Analysis

Các metric cần quan tâm:

- L1 accesses;
- L1 misses;
- L2 accesses;
- L2 misses;
- miss rate;
- latency;
- MSHR-related statistics nếu có;
- simulation / execution statistics.

6.16 Tổng hợp kết quả Miss Rate

Miss rate được tính:

$$MissRate = \frac{Misses}{Accesses} \tag{44}$$

Bảng kết quả:

Bảng 6: Miss rate theo cấu hình

Active ways	Accesses	Misses	Miss rate	So với baseline
16	measured	measured	calculated	0%
12	measured	measured	calculated	calculated
8	measured	measured	calculated	calculated
4	measured	measured	calculated	calculated
2	measured	measured	calculated	calculated

Theo yêu cầu đề bài, miss rate phải là số đo từ simulation hoặc được tính trực tiếp từ các statistics đo được.

6.17 Phân tích Miss Rate

Không chỉ ghi "miss rate tăng" mà cần giải thích nguyên nhân.

Các yếu tố cần xem xét:

- số active ways;
- cache geometry;
- set mapping;
- stride;
- footprint;
- locality;
- contention giữa các core.

Ví dụ, khi giảm active ways, cần kiểm tra xem sự thay đổi miss rate có phù hợp với address pattern và cache geometry hay không.

6.18 Tổng hợp kết quả AMAT

AMAT được tính sau khi đã thu thập miss rate và các latency cần thiết từ simulation hoặc từ cấu hình mô hình.

Với memory hierarchy L1–L2–DRAM, công thức sử dụng là:

$$AMAT_{system} = T_{L1} + MR_{L1}\left(T_{L2} + MR_{L2}P_{L2}\right) \tag{45}$$

Nếu mục tiêu chỉ là đánh giá riêng L2:

$$AMAT_{L2} = T_{L2} + MR_{L2}P_{L2} \tag{46}$$

P_{L2} phải được xác định theo cách đo của memory system.

Nếu latency được lấy trực tiếp từ statistics của gem5 thì phân loại là *measured*; nếu lấy từ configuration hoặc giả định của mô hình thì phải ghi rõ là *model*, *estimated* hoặc *assumed*.

Bảng 7: Nguồn dữ liệu sử dụng khi tính AMAT

Thông số	Phân loại	Nguồn dữ liệu
T_{L1}	Measured/Model	gem5 statistics/configuration
MR_{L1}	Measured/Calculated	gem5 statistics
T_{L2}	Measured/Model	gem5 statistics/configuration
MR_{L2}	Measured/Calculated	gem5 statistics
P_{L2}	Measured/Estimated	Memory system
$AMAT$	Calculated	Công thức AMAT

Không sử dụng một giá trị latency giả định như thể đó là số đo trực tiếp từ gem5.

Khi các thành phần latency không được gem5 cung cấp trực tiếp tại cùng một điểm đo, cần nêu rõ phương pháp tính và phạm vi của từng đại lượng.

6.19 So sánh hiệu năng

Sau khi có miss rate và AMAT, các configuration được so sánh với baseline.

Phần trăm thay đổi được tính theo:

$$\Delta Metric = \frac{Metric_{config} - Metric_{baseline}}{Metric_{baseline}} \times 100\% \tag{47}$$

Các tiêu chí cần đánh giá:

1. Giảm từ 16 xuống 12 way ảnh hưởng bao nhiêu?
2. Giảm xuống 8 way có còn chấp nhận được không?
3. Tại 4 hoặc 2 way, performance giảm mạnh ở đâu?
4. Capacity nào tạo ra trade-off tốt?
5. Associativity ảnh hưởng thế nào?
6. MSHR có trở thành bottleneck với workload hay không?

6.20 Phân tích hiệu năng

Vai trò Performance Analyst không chỉ là lấy một con số từ `stats.txt`.

Quy trình phân tích gồm:

$$Raw\ Statistics \rightarrow Metric \rightarrow Comparison \rightarrow Interpretation \rightarrow Engineering\ Conclusion \tag{48}$$

Cụ thể:

1. Xác định metric cần thu thập;
2. Thu thập statistics từ từng configuration;
3. Chuẩn hóa dữ liệu;
4. Tính miss rate;
5. Tính AMAT;
6. So sánh với baseline;
7. Xác định configuration tốt/xấu;
8. Giải thích nguyên nhân;
9. Rút ra trade-off.

6.21 Tổ chức và trình bày kết quả

Kết quả được tổ chức thành:

- bảng configuration;
- bảng miss rate;

- bảng AMAT;
- bảng MSHR;
- biểu đồ so sánh nếu cần;
- phần phân tích;
- phần giới hạn mô hình;
- kết luận kỹ thuật.

Mỗi kết quả phải được phân loại thành:

- **Measured**: lấy trực tiếp từ gem5;
- **Calculated**: tính từ measured data;
- **Estimated**: ước lượng;
- **Assumed**: giả định để phục vụ mô hình.

6.22 Giới hạn của mô hình

Báo cáo cần có một mục riêng về giới hạn mô hình.

1. `RiscvMinorCPU` là CPU model của gem5, không phải RTL implementation của CVA6.
2. Synthetic traffic chỉ xấp xỉ đặc tính của workload vector/DSP thực tế.
3. Miss rate phụ thuộc vào chuỗi địa chỉ và cache geometry.
4. Footprint lớn hơn L1 không đảm bảo mọi access đều là L1 miss.
5. Một số latency dùng cho AMAT có thể là model hoặc giả định thay vì số đo trực tiếp.
6. MSHR behavior phụ thuộc vào memory-level parallelism của workload.
7. Kết quả way shutdown phụ thuộc vào cách cache model được cấu hình để giới hạn active ways.